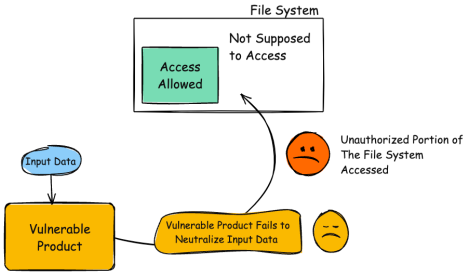
CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

Weakness ID: 22
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

The product uses external input to construct a pathname that is intended to identify a file or directory that is located underneath a restricted parent directory, but the product does not properly neutralize special elements within the pathname that can cause the pathname to resolve to a location that is outside of the restricted directory.



Extended Description

Many file operations are intended to take place within a restricted directory. By using special elements such as ".." and "/" separators, attackers can escape outside of the restricted location to access files or directories that are elsewhere on the system. One of the most common special elements is the "../" sequence, which in most modern operating systems is interpreted as the parent directory of the current location. This is referred to as relative path traversal. Path traversal also covers the use of absolute pathnames such as "/usr/local/bin" to access unexpected files. This is referred to as absolute path traversal.

Alternate Terms

Directory traversal	
Path traversal	"Path traversal" is preferred over "directory traversal," but both terms are attack-focused.

Common Consequences

Impact	Details
Execute Unauthorized Code or Commands	**Scope: Integrity, Confidentiality, Availability** The attacker may be able to create or overwrite critical files that are used to execute code, such as programs or libraries.
Modify Files or Directories	**Scope: Integrity** The attacker may be able to overwrite or create critical files, such as programs, libraries, or important data. If the targeted file is used for a security mechanism, then the attacker may be able to bypass that mechanism. For example, appending a new account at the end of a password file may allow an attacker to bypass authentication.
Read Files or Directories	**Scope: Confidentiality** The attacker may be able read the contents of unexpected files and expose sensitive data. If the targeted file is used for a security mechanism, then the attacker may be able to bypass that mechanism. For example, by reading a password file, the attacker could conduct brute force password guessing attacks in order to break into an account on the system.
DoS: Crash, Exit, or Restart	**Scope: Availability**

	The attacker may be able to overwrite, delete, or corrupt unexpected critical files such as programs, libraries, or important data. This may prevent the product from working at all and in the case of protection mechanisms such as authentication, it has the potential to lock out product users.

Potential Mitigations

Phase(s)	Mitigation
Implementation	**Strategy:** ***Input Validation*** Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use a list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if the input is only expected to contain colors such as "red" or "blue." Do not rely exclusively on looking for malicious or malformed inputs. This is likely to miss at least one undesirable input, especially if the code's environment changes. This can give attackers enough room to bypass the intended validation. However, denylists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When validating filenames, use stringent allowlists that limit the character set to be used. If feasible, only allow a single "." character in the filename to avoid weaknesses such as CWE-23, and exclude directory separators such as "/" to avoid CWE-36. Use a list of allowable file extensions, which will help to avoid CWE-434. Do not rely exclusively on a filtering mechanism that removes potentially dangerous characters. This is equivalent to a denylist, which may be incomplete (CWE-184). For example, filtering "/" is insufficient protection if the filesystem also supports the use of "\" as a directory separator. Another possible error could occur when the filtering is applied in a way that still produces dangerous data (CWE-182). For example, if "../" sequences are removed from the ".../.../" string in a sequential fashion, two instances of "../" would be removed from the original string, but the remaining characters would still form the "../" string.
Architecture and Design	For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server.
Implementation	**Strategy:** ***Input Validation*** Inputs should be decoded and canonicalized to the application's current internal representation before being validated (CWE-180). Make sure that the application does not decode the same input twice (CWE-174). Such errors could be used to bypass allowlist validation schemes by introducing dangerous inputs after they have been checked. Use a built-in path canonicalization function (such as realpath() in C) that produces the canonical version of the pathname, which effectively removes ".." sequences and symbolic links (CWE-23, CWE-59). This includes: • realpath() in C • getCanonicalPath() in Java • GetFullPath() in ASP.NET • realpath() or abs_path() in Perl • realpath() in PHP
Architecture and Design	**Strategy:** ***Libraries or Frameworks*** Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid [REF-1482].

Operation	**Strategy:** ***Firewall*** Use an application firewall that can detect attacks against this weakness. It can be beneficial in cases in which the code cannot be fixed (because it is controlled by a third party), as an emergency prevention measure while more comprehensive software assurance measures are applied, or to provide defense in depth [REF-1481]. **Effectiveness: Moderate** **Note:** An application firewall might not cover all possible input vectors. In addition, attack techniques might be available to bypass the protection mechanism, such as using malformed inputs that can still be processed by the component that receives those inputs. Depending on functionality, an application firewall might inadvertently reject or modify legitimate requests. Finally, some manual effort may be required for customization.
Architecture and Design; Operation	**Strategy:** ***Environment Hardening*** Run your code using the lowest privileges that are required to accomplish the necessary tasks [REF-76]. If possible, create isolated accounts with limited privileges that are only used for a single task. That way, a successful attack will not immediately give the attacker access to the rest of the software or its environment. For example, database applications rarely need to run as the database administrator, especially in day-to-day operations.
Architecture and Design	**Strategy:** ***Enforcement by Conversion*** When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input values (such as numeric IDs) to the actual filenames or URLs, and reject all other inputs. For example, ID 1 could map to "inbox.txt" and ID 2 could map to "profile.txt". Features such as the ESAPI AccessReferenceMap [REF-185] provide this capability.
Architecture and Design; Operation	**Strategy:** ***Sandbox or Jail*** Run the code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by the software. OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general, managed code may provide some protection. For example, java.io.FilePermission in the Java SecurityManager allows the software to specify restrictions on file operations. This may not be a feasible solution, and it only limits the impact to the operating system; the rest of the application may still be subject to compromise. Be careful to avoid CWE-243 and other weaknesses related to jails. **Effectiveness: Limited** **Note:** The effectiveness of this mitigation depends on the prevention capabilities of the specific sandbox or jail being used and might only help to reduce the scope of an attack, such as restricting the attacker to certain system calls or limiting the portion of the file system that can be accessed.
Architecture and Design; Operation	**Strategy:** ***Attack Surface Reduction*** Store library, include, and utility files outside of the web document root, if possible. Otherwise, store them in a separate directory and use the web server's access control capabilities to prevent attackers from directly requesting them. One common practice is to define a fixed constant in each calling program, then check for the existence of the constant in the library/include file; if the constant does not exist, then the file was directly requested, and it can exit immediately. This significantly reduces the chance of an attacker being able to bypass any protection mechanisms that are in the base program but not in the include files. It will also reduce the attack surface.
Implementation	Ensure that error messages only contain minimal details that are useful to the intended audience and no one else. The messages need to strike the balance between being too cryptic (which can confuse users) or being too detailed (which may reveal more than intended). The messages should not reveal the methods that were used to determine the error. Attackers can use detailed information to refine or optimize their original attack, thereby increasing their chances of success. If errors must be captured in some detail, record them in log messages, but consider what could occur if the log messages can be viewed by attackers. Highly

	sensitive information such as passwords should never be saved to log files. Avoid inconsistent messaging that might accidentally tip off an attacker about internal state, such as whether a user account exists or not. In the context of path traversal, error messages which disclose path information can help attackers craft the appropriate attack strings to move through the file system hierarchy.
Operation; Implementation	**Strategy:** ***Environment Hardening*** When using PHP, configure the application so that it does not use register_globals. During implementation, develop the application so that it does not rely on this feature, but be wary of implementing a register_globals emulation that is subject to weaknesses such as CWE-95, CWE-621, and similar issues.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf	C	706	Use of Incorrectly-Resolved Name or Reference
ParentOf	B	23	Relative Path Traversal
ParentOf	B	36	Absolute Path Traversal
CanFollow	C	20	Improper Input Validation
CanFollow	B	73	External Control of File Name or Path
CanFollow	C	172	Encoding Error
CanPrecede	C	668	Exposure of Resource to Wrong Sphere

Relevant to the view "Software Development" (View-699)

Nature	Type	ID	Name
MemberOf	C	1219	File Handling Issues

Relevant to the view "Weaknesses for Simplified Mapping of Published Vulnerabilities" (View-1003)

Nature	Type	ID	Name
ChildOf	C	706	Use of Incorrectly-Resolved Name or Reference

Relevant to the view "CISQ Quality Measures (2020)" (View-1305)

Nature	Type	ID	Name
ParentOf	B	23	Relative Path Traversal
ParentOf	B	36	Absolute Path Traversal

Relevant to the view "CISQ Data Protection Measures" (View-1340)

Nature	Type	ID	Name
ParentOf	B	23	Relative Path Traversal
ParentOf	B	36	Absolute Path Traversal

Modes Of Introduction

Phase	Note
Implementation	

Applicable Platforms

Languages	Class: Not Language-Specific *(Undetermined Prevalence)*
Technologies	AI/ML *(Undetermined Prevalence)*

Likelihood Of Exploit

High

Demonstrative Examples

Example 1

The following code could be for a social networking application in which each user's profile information is stored in a separate file. All files are stored in a single directory.

Example Language: **Perl** *(bad code)*

```
my $dataPath = "/users/cwe/profiles";
my $username = param("user");
my $profilePath = $dataPath . "/" . $username;

open(my $fh, "<", $profilePath) || ExitError("profile read error: $profilePath");
print "<ul>\n";
while (<$fh>) {
  print "<li>$_</li>\n";
}
print "</ul>\n";
```

While the programmer intends to access files such as "/users/cwe/profiles/alice" or "/users/cwe/profiles/bob", there is no verification of the incoming user parameter. An attacker could provide a string such as:

(attack code)

```
../../../etc/passwd
```

The program would generate a profile pathname like this:

(result)

```
/users/cwe/profiles/../../../etc/passwd
```

When the file is opened, the operating system resolves the "../" during path canonicalization and actually accesses this file:

(result)

```
/etc/passwd
```

As a result, the attacker could read the entire text of the password file.

Notice how this code also contains an error message information leak (CWE-209) if the user parameter does not produce a file that exists: the full pathname is provided. Because of the lack of output encoding of the file that is retrieved, there might also be a cross-site scripting problem (CWE-79) if profile contains any HTML, but other code would need to be examined.

Example 2

In the example below, the path to a dictionary file is read from a system property and used to initialize a File object.

Example Language: **Java** *(bad code)*

```
String filename = System.getProperty("com.domain.application.dictionaryFile");
File dictionaryFile = new File(filename);
```

However, the path is not validated or modified to prevent it from containing relative or absolute path sequences before creating the File object. This allows anyone who can control the system property to determine what file is used. Ideally, the path should be resolved relative to some kind of application or user home directory.

Example 3

The following code takes untrusted input and uses a regular expression to filter "../" from the input. It then appends this result to the /home/user/ directory and attempts to read the file in the final resulting path.

Example Language: **Perl** *(bad code)*

```
my $Username = GetUntrustedInput();
$Username =~ s/\.\.\///;
```

```
my $filename = "/home/user/" . $username;
ReadAndSendFile($filename);
```

Since the regular expression does not have the /g global match modifier, it only removes the first instance of "../" it comes across. So an input value such as:

(attack code)

```
../../../etc/passwd
```

will have the first "../" stripped, resulting in:

(result)

```
../../etc/passwd
```

This value is then concatenated with the /home/user/ directory:

(result)

```
/home/user/../../etc/passwd
```

which causes the /etc/passwd file to be retrieved once the operating system has resolved the ../ sequences in the pathname. This leads to relative path traversal (CWE-23).

Example 4

The following code attempts to validate a given input path by checking it against an allowlist and once validated delete the given file. In this specific case, the path is considered valid if it starts with the string "/safe_dir/".

Example Language: **Java** *(bad code)*

```
String path = getInputPath();
if (path.startsWith("/safe_dir/"))
{
  File f = new File(path);
  f.delete()
}
```

An attacker could provide an input such as this:

(attack code)

```
/safe_dir/../important.dat
```

The software assumes that the path is valid because it starts with the "/safe_path/" sequence, but the "../" sequence will cause the program to delete the important.dat file in the parent directory

---

Example 5

The following code demonstrates the unrestricted upload of a file with a Java servlet and a path traversal vulnerability. The action attribute of an HTML form is sending the upload file request to the Java servlet.

Example Language: **HTML** *(good code)*

```
<form action="FileUploadServlet" method="post" enctype="multipart/form-data">

Choose a file to upload:
<input type="file" name="filename"/>
<br/>
<input type="submit" name="submit" value="Submit"/>

</form>
```

When submitted the Java servlet's doPost method will receive the request, extract the name of the file from the Http request header, read the file contents from the request and output the file to the local upload directory.

Example Language: **Java** *(bad code)*

```
public class FileUploadServlet extends HttpServlet {
  ...

  protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
  ServletException, IOException {
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    String contentType = request.getContentType();

    // the starting position of the boundary header
    int ind = contentType.indexOf("boundary=");
    String boundary = contentType.substring(ind+9);

    String pLine = new String();
    String uploadLocation = new String(UPLOAD_DIRECTORY_STRING); //Constant value

    // verify that content type is multipart form data
    if (contentType != null && contentType.indexOf("multipart/form-data") != -1) {
      // extract the filename from the Http header
      BufferedReader br = new BufferedReader(new InputStreamReader(request.getInputStream()));
      ...
      pLine = br.readLine();
      String filename = pLine.substring(pLine.lastIndexOf("\\"), pLine.lastIndexOf("\""));
      ...

      // output the file to the local upload directory
      try {
        BufferedWriter bw = new BufferedWriter(new FileWriter(uploadLocation+filename, true));
        for (String line; (line=br.readLine())!=null; ) {
          if (line.indexOf(boundary) == -1) {
            bw.write(line);
            bw.newLine();
            bw.flush();
          }
        } //end of for loop
        bw.close();


      } catch (IOException ex) {...}
      // output successful upload response HTML page
    }
    // output unsuccessful upload response HTML page
    else
    {...}
  }
    ...
}
```

This code does not perform a check on the type of the file being uploaded (CWE-434). This could allow an attacker to upload any executable file or other file with malicious code.

Additionally, the creation of the BufferedWriter object is subject to relative path traversal (CWE-23). Since the code does not check the filename that is provided in the header, an attacker can use "../" sequences to write to files outside of the intended directory. Depending on the executing environment, the attacker may be able to specify arbitrary files to write to, leading to a wide variety of consequences, from code execution, XSS (CWE-79), or system crash.

Example 6

This script intends to read a user-supplied file from the current directory. The user inputs the relative path to the file and the script uses Python's os.path.join() function to combine the path to the current working directory with the provided path to the specified file. This results in an absolute path to the desired file. If the file does not exist when the script attempts to read it, an error is printed to the user.

Example Language: **Python** *(bad code)*

```
import os
import sys
```

```
def main():
  filename = sys.argv[1]
  path = os.path.join(os.getcwd(), filename)
  try:
    with open(path, 'r') as f:
      file_data = f.read()
  except FileNotFoundError as e:
    print("Error - file not found")
main()
```

However, if the user supplies an absolute path, the os.path.join() function will discard the path to the current working directory and use only the absolute path provided. For example, if the current working directory is /home/user/documents, but the user inputs /etc/passwd, os.path.join() will use only /etc/passwd, as it is considered an absolute path. In the above scenario, this would cause the script to access and read the /etc/passwd file.

Example Language: **Python** *(good code)*

```
import os
import sys
def main():
  filename = sys.argv[1]
  path = os.path.normpath(f"{os.getcwd()}{os.sep}{filename}")
  if path.startswith("/home/cwe/documents/"):
    try:
      with open(path, 'r') as f:
        file_data = f.read()
    except FileNotFoundError as e:
      print("Error - file not found")
main()
```

The constructed path string uses os.sep to add the appropriate separation character for the given operating system (e.g. '\' or '/') and the call to os.path.normpath() removes any additional slashes that may have been entered - this may occur particularly when using a Windows path. The path is checked against an expected directory (/home/cwe/documents); otherwise, an attacker could provide relative path sequences like ".." to cause normpath() to generate paths that are outside the intended directory (CWE-23). By putting the pieces of the path string together in this fashion, the script avoids a call to os.path.join() and any potential issues that might arise if an absolute path is entered. With this version of the script, if the current working directory is /home/cwe/documents, and the user inputs /etc/passwd, the resulting path will be /home/cwe/documents/etc/passwd. The user is therefore contained within the current working directory as intended.

Selected Observed Examples

Note: this is a curated list of examples for users to understand the variety of ways in which this weakness can be introduced. It is not a complete list of all CVEs that are related to this CWE entry.

Reference	Description
CVE-2024-37032	Large language model (LLM) management tool does not validate the format of a digest value (CWE-1287) from a private, untrusted model registry, enabling relative path traversal (CWE-23), a.k.a. Probllama
CVE-2024-4315	Chain: API for text generation using Large Language Models (LLMs) does not include the "\" Windows folder separator in its denylist (CWE-184) when attempting to prevent Local File Inclusion via path traversal (CWE-22), allowing deletion of arbitrary files on Windows systems.
CVE-2024-0520	Product for managing datasets for AI model training and evaluation allows both relative (CWE-23) and absolute (CWE-36) path traversal to overwrite files via the Content-Disposition header
CVE-2022-45918	Chain: a learning management tool debugger uses external input to locate previous session logs (CWE-73) and does not properly validate the given path (CWE-20), allowing for filesystem path traversal using "../" sequences (CWE-24)
CVE-2019-20916	Python package manager does not correctly restrict the filename specified in a Content-Disposition header, allowing arbitrary file read using path traversal sequences such as "../"
CVE-2022-31503	Python package constructs filenames using an unsafe os.path.join call on untrusted input, allowing absolute path traversal because os.path.join resets the pathname to an

	absolute path that is specified as part of the input.
CVE-2022-24877	directory traversal in Go-based Kubernetes operator app allows accessing data from the controller's pod file system via ../ sequences in a yaml file
CVE-2021-21972	Chain: Cloud computing virtualization platform does not require authentication for upload of a tar format file (CWE-306), then uses .. path traversal sequences (CWE-23) in the file to access unexpected files, as exploited in the wild per CISA KEV.
CVE-2020-4053	a Kubernetes package manager written in Go allows malicious plugins to inject path traversal sequences into a plugin archive ("Zip slip") to copy a file outside the intended directory
CVE-2020-3452	Chain: security product has improper input validation (CWE-20) leading to directory traversal (CWE-22), as exploited in the wild per CISA KEV.
CVE-2019-10743	Go-based archive library allows extraction of files to locations outside of the target folder with "../" path traversal sequences in filenames in a zip file, aka "Zip Slip"
CVE-2010-0467	Newsletter module allows reading arbitrary files using "../" sequences.
CVE-2006-7079	Chain: PHP app uses extract for register_globals compatibility layer (CWE-621), enabling path traversal (CWE-22)
CVE-2009-4194	FTP server allows deletion of arbitrary files using ".." in the DELE command.
CVE-2009-4053	FTP server allows creation of arbitrary directories using ".." in the MKD command.
CVE-2009-0244	FTP service for a Bluetooth device allows listing of directories, and creation or reading of files using ".." sequences.
CVE-2009-4013	Software package maintenance program allows overwriting arbitrary files using "../" sequences.
CVE-2009-4449	Bulletin board allows attackers to determine the existence of files using the avatar.
CVE-2009-4581	PHP program allows arbitrary code execution using ".." in filenames that are fed to the include() function.
CVE-2010-0012	Overwrite of files using a .. in a Torrent file.
CVE-2010-0013	Chat program allows overwriting files using a custom smiley request.
CVE-2008-5748	Chain: external control of values for user's desired language and theme enables path traversal.
CVE-2009-1936	Chain: library file sends a redirect if it is directly requested but continues to execute, allowing remote file inclusion and path traversal.

Weakness Ordinalities

Ordinality	Description
Primary	*(where the weakness exists independent of other weaknesses)*
Resultant	*(where the weakness is typically related to the presence of some other weaknesses)*

Detection Methods

Method	Details
Automated Static Analysis	Automated techniques can find areas where path traversal weaknesses exist. However, tuning or customization may be required to remove or de-prioritize path-traversal problems that are only exploitable by the product's administrator - or other privileged users - and thus potentially valid behavior or, at worst, a bug instead of a vulnerability. **Effectiveness: High**
Manual Static Analysis	Manual white box techniques may be able to provide sufficient code coverage and reduction of false positives if all file access operations can be assessed within limited time constraints.

	Effectiveness: High
Automated Static Analysis - Binary or Bytecode	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Bytecode Weakness Analysis - including disassembler + source code weakness analysis Cost effective for partial coverage: • Binary Weakness Analysis - including disassembler + source code weakness analysis **Effectiveness: High**
Manual Static Analysis - Binary or Bytecode	According to SOAR [REF-1479], the following detection techniques may be useful: Cost effective for partial coverage: • Binary / Bytecode disassembler - then use manual analysis for vulnerabilities & anomalies **Effectiveness: SOAR Partial**
Dynamic Analysis with Automated Results Interpretation	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Web Application Scanner • Web Services Scanner • Database Scanners **Effectiveness: High**
Dynamic Analysis with Manual Results Interpretation	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Fuzz Tester • Framework-based Fuzzer **Effectiveness: High**
Manual Static Analysis - Source Code	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Manual Source Code Review (not inspections) Cost effective for partial coverage: • Focused Manual Spotcheck - Focused manual analysis of source **Effectiveness: High**
Automated Static Analysis - Source Code	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Source code Weakness Analyzer

	• Context-configured Source Code Weakness Analyzer **Effectiveness: High**
Architecture or Design Review	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Formal Methods / Correct-By-Construction Cost effective for partial coverage: • Inspection (IEEE 1028 standard) (can apply to requirements, design, source code, etc.) **Effectiveness: High**

Functional Areas

- File Processing

Affected Resources

- File or Directory

Memberships

Nature	Type	ID	Name
MemberOf	V	635	Weaknesses Originally Used by NVD from 2008 to 2016
MemberOf	C	715	OWASP Top Ten 2007 Category A4 - Insecure Direct Object Reference
MemberOf	C	723	OWASP Top Ten 2004 Category A2 - Broken Access Control
MemberOf	C	743	CERT C Secure Coding Standard (2008) Chapter 10 - Input Output (FIO)
MemberOf	C	802	2010 Top 25 - Risky Resource Management
MemberOf	C	813	OWASP Top Ten 2010 Category A4 - Insecure Direct Object References
MemberOf	C	865	2011 Top 25 - Risky Resource Management
MemberOf	C	877	CERT C++ Secure Coding Section 09 - Input Output (FIO)
MemberOf	V	884	CWE Cross-section
MemberOf	C	932	OWASP Top Ten 2013 Category A4 - Insecure Direct Object References
MemberOf	C	981	SFP Secondary Cluster: Path Traversal
MemberOf	C	1031	OWASP Top Ten 2017 Category A5 - Broken Access Control
MemberOf	C	1131	CISQ Quality Measures (2016) - Security
MemberOf	C	1179	SEI CERT Perl Coding Standard - Guidelines 01. Input Validation and Data Sanitization (IDS)
MemberOf	V	1200	Weaknesses in the 2019 CWE Top 25 Most Dangerous Software Errors
MemberOf	C	1308	CISQ Quality Measures - Security
MemberOf	V	1337	Weaknesses in the 2021 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1340	CISQ Data Protection Measures
MemberOf	C	1345	OWASP Top Ten 2021 Category A01:2021 - Broken Access Control
MemberOf	V	1350	Weaknesses in the 2020 CWE Top 25 Most Dangerous Software Weaknesses

MemberOf	V	1387	Weaknesses in the 2022 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	C	1404	Comprehensive Categorization: File Handling
MemberOf	V	1425	Weaknesses in the 2023 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1430	Weaknesses in the 2024 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1435	Weaknesses in the 2025 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	C	1436	OWASP Top Ten 2025 Category A01:2025 - Broken Access Control

Vulnerability Mapping Notes

Usage	**ALLOWED-WITH-REVIEW** *(this CWE ID could be used to map to real-world vulnerabilities in limited situations requiring careful review)*
Reason	Abstraction
Rationale	This CWE entry might have children that would be more appropriate.
Comments	Examine children of this entry to see if there is a better fit. Consider children such as CWE-23 (or its descendants) for relative path traversal, or CWE-36 for absolute path traversal.
Suggestions	**CWE-ID** / **Comment** CWE-23 — relative path traversal - also consider descendants CWE-36 — absolute path traversal

Notes

Relationship

Pathname equivalence can be regarded as a type of canonicalization error.

Relationship

Some pathname equivalence issues are not directly related to directory traversal, rather are used to bypass security-relevant checks for whether a file/directory can be accessed by the attacker (e.g. a trailing "/" on a filename could bypass access rules that don't expect a trailing /, causing a server to provide the file when it normally would not).

Terminology

Like other weaknesses, terminology is often based on the types of manipulations used, instead of the underlying weaknesses. Some people use "directory traversal" only to refer to the injection of ".." and equivalent sequences whose specific meaning is to traverse directories.

Other variants like "absolute pathname" and "drive letter" have the *effect* of directory traversal, but some people may not call it such, since it doesn't involve ".." or equivalent.

Research Gap

Many variants of path traversal attacks are probably under-studied with respect to root cause. CWE-790 and CWE-182 begin to cover part of this gap.

Research Gap

Incomplete diagnosis or reporting of vulnerabilities can make it difficult to know which variant is affected. For example, a researcher might say that "..\" is vulnerable, but not test "../" which may also be vulnerable.

Any combination of directory separators ("/", "\", etc.) and numbers of "." (e.g. "....") can produce unique variants; for example, the "//../" variant is not listed (CVE-2004-0325). See this entry's children and lower-level descendants.

Other

In many programming languages, the injection of a null byte (the 0 or NUL) may allow an attacker to truncate a generated filename to apply to a wider range of files. For example, the product may add ".txt" to any pathname, thus limiting the attacker to text files, but a null injection may effectively remove this restriction.

Taxonomy Mappings

Mapped Taxonomy Name	Node ID	Fit	Mapped Node Name

PLOVER			Path Traversal
OWASP Top Ten 2007	A4	CWE More Specific	Insecure Direct Object Reference
OWASP Top Ten 2004	A2	CWE More Specific	Broken Access Control
CERT C Secure Coding	FIO02-C		Canonicalize path names originating from untrusted sources
SEI CERT Perl Coding Standard	IDS00-PL	Exact	Canonicalize path names before validating them
WASC	33		Path Traversal
Software Fault Patterns	SFP16		Path Traversal
OMG ASCSM	ASCSM-CWE-22		

Related Attack Patterns

CAPEC-ID	Attack Pattern Name
CAPEC-126	Path Traversal
CAPEC-64	Using Slashes and URL Encoding Combined to Bypass Validation Logic
CAPEC-76	Manipulating Web Input to File System Calls
CAPEC-78	Using Escaped Slashes in Alternate Encoding
CAPEC-79	Using Slashes in Alternate Encoding

References

[REF-7] Michael Howard and David LeBlanc. *"Writing Secure Code".* Chapter 11, "Directory Traversal and Using Parent Paths (..)" Page 370. 2nd Edition. Microsoft Press. 2002-12-04. <https://www.microsoftpressstore.com/store/writing-secure-code-9780735617223>.

[REF-45] OWASP. *"OWASP Enterprise Security API (ESAPI) Project".* <https://owasp.org/www-project-enterprise-security-api/>. (*URL validated: 2025-07-24*)

[REF-185] OWASP. *"Testing for Path Traversal (OWASP-AZ-001)".* <http://www.owasp.org/index.php/Testing_for_Path_Traversal_(OWASP-AZ-001)>.

[REF-186] Johannes Ullrich. *"Top 25 Series - Rank 7 - Path Traversal".* SANS Software Security Institute. 2010-03-09. <https://www.sans.org/blog/top-25-series-rank-7-path-traversal/>. (*URL validated: 2023-04-07*)

[REF-76] Sean Barnum and Michael Gegick. *"Least Privilege".* 2005-09-14. <https://web.archive.org/web/20211209014121/https://www.cisa.gov/uscert/bsi/articles/knowledge/principles/least-privilege>. (*URL validated: 2023-04-07*)

[REF-62] Mark Dowd, John McDonald and Justin Schuh. *"The Art of Software Security Assessment".* Chapter 9, "Filenames and Paths", Page 503. 1st Edition. Addison Wesley. 2006.

[REF-962] Object Management Group (OMG). *"Automated Source Code Security Measure (ASCSM)".* ASCSM-CWE-22. 2016-01. <http://www.omg.org/spec/ASCSM/1.0/>.

[REF-1448] Cybersecurity and Infrastructure Security Agency. *"Secure by Design Alert: Eliminating Directory Traversal Vulnerabilities in Software".* 2024-05-02. <https://www.cisa.gov/resources-tools/resources/secure-design-alert-eliminating-directory-traversal-vulnerabilities-software>. (*URL validated: 2024-07-14*)

[REF-1479] Gregory Larsen, E. Kenneth Hong Fong, David A. Wheeler and Rama S. Moorthy. *"State-of-the-Art Resources (SOAR) for Software Vulnerability Detection, Test, and Evaluation".* 2014-07. <https://www.ida.org/-/media/feature/publications/s/st/stateoftheart-resources-soar-for-

	software-vulnerability-detection-test-and-evaluation/p-5061.ashx>. (*URL validated: 2025-09-05*)
[REF-1481]	D3FEND. *"D3FEND: Application Layer Firewall".* <https://d3fend.mitre.org/dao/artifact/d3f:ApplicationLayerFirewall/>. (*URL validated: 2025-09-06*)
[REF-1482]	D3FEND. *"D3FEND: D3-TL Trusted Library".* <https://d3fend.mitre.org/technique/d3f:TrustedLibrary/>. (*URL validated: 2025-09-06*)

Content History

Submissions

Submission Date	Submitter	Organization
2006-07-19 *(CWE Draft 3, 2006-07-19)*	PLOVER	

Contributions

Contribution Date	Contributor	Organization
2022-07-11	Nick Johnston *Identified weakness in Perl demonstrative example*	
2024-02-29 *(CWE 4.15, 2024-07-16)*	Abhi Balakrishnan *Provided diagram to improve CWE usability*	
2024-11-01	Drew Buttner *Identified weakness in "good code" for Python demonstrative example*	MITRE

Modifications

Previous Entry Names